

Invariant-Preserving Compression of Structured Tool Outputs for LLM Agents

Pramod Ubbala

Independent Researcher

ORCID: 0009-0005-5816-2998

Corresponding author: Pramod Ubbala, ubbalap@gmail.com

Research Article

Abstract

Tool-using large language model agents repeatedly ingest JSON, logs, search results, and API responses. The resulting context can dominate input size, but aggressive reduction creates a harder failure mode than ordinary summarization: a compressor can remove one identifier, timestamp, numeric field, or record and still produce text that looks plausible. We introduce an invariant-preserving compression gate for structured tool outputs. The gate accepts any candidate compressor, computes executable task invariants on the original and compressed payloads, and releases the compressed result only when those invariants match. A failed check triggers contract-aware recovery or a fail-open return of the original payload. We evaluate 960 seeded cases spanning six workload domains, eight task families, and payloads of 50 to 500 records. The gate reduced serialized context by 90.23% on average while preserving every benchmark answer and invariant. Across nine candidate-aggressiveness settings, preservation remained 100% while mean reduction ranged from 84.83% to 92.21%. A case-level bootstrap placed the reported configuration's mean reduction at 89.42% to 91.00% with 95% confidence. An unverified task-aware reducer removed 99.76% but changed 21.46% of answers. A study-owned pattern-retention guard preserved every seeded answer but reduced context by 70.51%. On a second validation using 24 public GitHub issue records and eight issue-tracker tasks, the invariant gate reduced context by 81.47% with 100% answer and invariant preservation. The pattern-retention guard reduced context by 26.42% and preserved 87.50% of answers. All 900 injected contract-relevant corruptions were detected. The result marks a practical distinction: protecting known row matches is useful, but task semantics such as negative evidence, exact aggregation, temporal maxima, and cross-record relations require executable contracts over the structured result.

Keywords: LLM agents; context compression; tool outputs; semantic invariants; JSON; reliability

1. Introduction

Agentic LLM applications accumulate context quickly. A single reasoning step can add hundreds or thousands of structured records from search, observability, databases, ticketing systems, security scanners, or identity services. Long-context models postpone the hard limit, but they do not eliminate the cost of repeatedly sending low-value fields and redundant rows. Prompt-compression research has therefore moved from token pruning toward long-context and agent-state reduction [1-5].

Structured tool output creates a distinct problem. One missing word in prose may only weaken a summary. One missing identifier in JSON can change which object the agent acts on. A dropped timestamp can reverse the apparent order of events. Removing a low-frequency error can turn an incident into a healthy system. A compressed payload may remain syntactically valid while becoming semantically wrong for the task that follows.

Existing work exposes this tension. LLMingua and LongLLMingua reduce natural-language prompts through learned token importance [2,3]. ProCut prunes prompt components through attribution [5]. ACON compresses long-horizon agent observations and histories by optimizing natural-language compression guidelines from failure examples [7]. TRACE tests summary quality through paired closed-loop continuations and reports execution instability after recurrent compaction [8]. ToolLLM addressed long API responses earlier by learning response-field pruning strategies for fixed APIs [9]. Headroom 0.36.0 includes protected-pattern row retention and a fail-closed check for configured matches in its audit-safe SmartCrusher path [13,16,17]. The remaining gap is narrower than simple preservation. Pattern retention does not directly express exact aggregates, negative evidence, temporal maxima, cross-record relations, or field-level provenance as task-equivalence obligations, and it binds protection to predicates understood by the compressor rather than to an external contract over arbitrary candidates.

This paper targets that boundary. We treat compression as an untrusted optimization. The application declares a small semantic contract over a tool result, such as identity preservation, latest-state preservation, exact aggregation, rare-event completeness, threshold completeness, or provenance retention. The compressor produces a candidate payload. A deterministic gate compares the contract invariants before and after compression. Matching invariants release the candidate. A mismatch invokes a contract-aware recovery transform; if recovery still fails, the gate returns the original payload.

The design borrows the discipline of software contracts rather than asking a language model to judge whether its own context remains adequate. Design by Contract established the value of executable preconditions, postconditions, and invariants for software correctness decades ago [10]. Here the same idea is placed at the context boundary of an LLM agent. The contract does not claim that two prompts are linguistically equivalent. It states which structured facts must remain equivalent for a declared task.

We make five contributions. First, we define a task-semantic contract for structured tool outputs and separate it from the compressor itself. Second, we implement an accept, recover, or fail-open gate that can sit in front of any LLM provider or candidate compressor. Third, we construct a seeded benchmark with 960 cases across six operational domains and eight task families, then inject 900 targeted corruptions to test the verifier. Fourth, we compare fixed-field projection, head-tail sampling, anomaly sampling, an unverified task-aware reducer, and a study-owned protected-pattern row-retention guard against the invariant gate. Fifth, we repeat the core comparison on a small public issue-tracker snapshot and test the gate across nine candidate-aggressiveness settings with interval estimates for the principal preservation and reduction measurements. The strongest result is not the largest reduction. It is the absence of silent benchmark-answer changes under the declared contracts, including task shapes that positive row-pattern matching does not directly encode.

2. Related Work and Research Gap

2.1 Prompt and long-context compression

LLMingua introduced coarse-to-fine prompt compression using a small language model, a budget controller, and iterative token-level pruning, reporting up to 20x compression with limited performance loss on its evaluated tasks [2]. LongLLMingua extended this line to long-context settings and reported strong cost and latency reductions while attempting to preserve key information positions [3]. 500xCompressor pushed compression into learned special-token representations and reported ratios from 6x to 500x, trading some downstream capability for much

smaller context representations [4]. ProCut takes a different path: it estimates the contribution of semantically meaningful prompt units and removes low-utility sections without model training [5].

These methods primarily reason about prompts or natural-language context. They are valuable baselines for the broader efficiency problem, but their safety signal is normally downstream task performance, model attribution, or information-retention metrics. A structured API payload often permits a stronger check. The application can compute exact properties before the text reaches the LLM.

2.2 Information preservation and agent context

Information loss has become a first-class evaluation issue. Łajewska et al. showed that prompt compressors can lose key details even when aggregate downstream metrics appear acceptable, and they evaluated grounding and entity preservation alongside task performance [6]. ACON moved compression into long-horizon agents, covering environment observations and interaction histories rather than isolated prompts [7]. TRACE then focused on recurrent compaction failures, reporting blocked actions, repeated exploration, and run-to-run instability after summaries weaken recent interaction state [8]. Recent production-oriented preprints also warn that aggressive input reduction can increase output length or total inference cost, so input tokens alone are not a sufficient deployment metric [11,12].

Our scope is narrower. We do not summarize full trajectories, compress chain-of-thought, or learn a natural-language memory policy. We isolate runtime structured tool outputs. This narrower scope permits deterministic verification before inference.

2.3 Structured API response reduction

ToolLLM recognized the API-response problem in 2023. Its ToolBench pipeline used ChatGPT to inspect a sample response for each API and remove unimportant keys; responses still above a fixed length were truncated [9]. That strategy is efficient when response formats are stable, but a fixed schema-level decision cannot see which fields become critical for a particular downstream question.

Headroom is a current open-source practitioner system aimed directly at context reduction for tool outputs, logs, RAG chunks, files, and conversation history [13]. Its documentation reports large reductions on repetitive JSON and log workloads, and its SmartCrusher strategy preserves boundaries, anomalies, and statistical structure for arrays [13,14]. Headroom 0.36.0 also exposes an audit-safe mode for JSON-array-shaped content [16,17]. A caller supplies protected string or regular-expression patterns. The Python layer scans canonical row JSON before compression, splices back protected rows removed by the Rust-backed selector, re-counts protected survivors, and can return the original payload if protected matches remain missing [17]. This is close in spirit to the safety objective studied here and narrows our claim. Headroom protects configured row matches. The proposed gate verifies arbitrary deterministic task invariants over the original and candidate payloads, including exact aggregates, negative evidence, latest-state semantics, cross-record relations, thresholds, and provenance signatures. Section 5 therefore adds a study-owned pattern-retention guard inspired by these published semantics. It is not Headroom code and does not reproduce Headroom's statistical selector, compaction, or retrieval system. We do not report a local Headroom runtime result because the native wheel could not cross the execution environment's package-download boundary. The secondary validation uses public Headroom GitHub issue metadata only as a non-synthetic structured dataset [15].

Table 1. Position of this work relative to selected compression approaches.

Approach	Primary unit	Safety or utility signal	Structured tool-output focus	Runtime fail-open
LLMLingua [2]	Prompt tokens	Model-estimated importance and downstream task score	No	No
LongLLMLingua [3]	Long prompt tokens	Key-information retention and downstream task score	No	No
ProCut [5]	Prompt sections	Attribution to task performance	No	No
ToolLLM [9]	API response fields	Static per-API schema pruning plus human evaluation	Yes	No
ACON [7]	Agent observations and history	Failure-guided compression policy	Partial	No
TRACE [8]	Agent summaries	Paired closed-loop continuation	Partial	Boundary verification
Headroom [13,14,16,17]	Tool outputs and context blocks	Content heuristics, protected-pattern row retention, accuracy tests, reversible retrieval	Yes	Yes for protected matches
This work	Structured tool outputs	Executable task-equivalence invariants external to candidate compressor	Yes	Yes for contract failure

3. Problem Definition

Let x denote an original structured tool result, q the downstream task, C an arbitrary lossy compressor, and $y = C(x)$ the candidate compressed result. Conventional compression selects y according to a size or relevance objective. We add a contract $K(q)$ that defines the semantic properties the application refuses to lose.

A contract contains one or more executable invariant functions I_j . Examples include the complete set of failed record identifiers, the latest timestamp and state for a target entity, the exact sum and count of a numeric field within a category, all records above a threshold, or the provenance URI attached to every critical finding. The invariants may use fields that appear unimportant under global statistics. Their importance comes from the task.

The gate accepts y only when every declared invariant matches the original payload.

$$G(x, y, K) = 1 \text{ iff } I_j(x; K) = I_j(y; K) \text{ for every invariant } j \text{ in } K.$$

A rejected candidate does not immediately force the full original context. The recovery transform $R_K(x)$ selects the rows required by the contract and projects only the fields needed to recompute the same invariants. The gate verifies $R_K(x)$ again. If the recovered payload fails, the system fails open to x .

The optimization target is therefore constrained reduction rather than unconstrained minimization.

$$\text{minimize } \text{size}(y) \text{ subject to } G(x, y, K) = 1.$$

The implementation measures exact UTF-8 serialized bytes. We use bytes because the local environment did not contain a production tokenizer and could not download one. The paper therefore does not claim that the reported percentage is an exact token-saving percentage for any specific model. Field and row removal normally reduces tokens as well, but tokenizer-specific measurement belongs in the next external replication.

3.1 Threat model

The compressor is treated as fallible, not malicious. It may prune a rare row, drop a field, retain the wrong temporal boundary, or reduce a collection in a way that destroys evidence of absence. The gate assumes that the original tool result is available at verification time and that the contract implementation is trusted application code.

The method does not defend against an incorrect original tool response, a malicious tool that lies before compression, or a contract that omits the fact actually needed by the downstream task. Contract quality is therefore the central trust assumption. That limitation is explicit rather than hidden behind a similarity score.

4. Invariant-Preserving Compression Gate

4.1 Contract classes

The prototype supports eight contract classes chosen because they recur in enterprise tool use and fail under different compression patterns:

Contract class	Invariant obligation
Identity lookup	Preserve the exact target record and task-required fields.
Rare-event completeness	Preserve every record matching a rare state or predicate.
Latest-state correctness	Preserve enough events to identify the true maximum timestamp for a target entity and its state.
Aggregation integrity	Preserve the record population and numeric values required for exact count and sum.
Cross-record relation	Preserve every row participating in the declared relation key.
Negative evidence	Preserve the full scoped population required to prove that a forbidden state is absent.
Threshold completeness	Preserve every record whose metric satisfies a declared threshold predicate.
Provenance retention	Preserve source and evidence identifiers for all records in the declared evidence class.

4.2 Gate execution

The gate is intentionally small. It has no model call, embedding model, or learned parameter. A candidate compressor can be changed without changing the verification boundary.

```

Algorithm 1: Invariant-preserving compression gate
input: original x, task contract K, candidate compressor C
candidate = C(x)
reference = invariants(x, K)
if invariants(candidate, K) == reference:
    return candidate
recovered = contract_aware_projection(x, K)
if invariants(recovered, K) == reference:
    return recovered
return x # fail open

```

The candidate path can retain a compressor's aggressive reduction when it happens to preserve the required semantics. Recovery pays extra work only when the candidate violates a contract. This separation matters operationally. The gate is not a new universal compressor. It is a correctness boundary around compressors.

4.3 Recovery transform

Recovery uses the contract to identify rows and fields that are sufficient for the task family. Lookup tasks retain the target row only. Rare-event, threshold, and provenance tasks retain all predicate matches. Aggregations retain every in-scope row but only the grouping and numeric fields required by the calculation. Latest-state contracts retain all events for the target entity before projecting their identifier, timestamp, and state. Negative-evidence contracts retain every row in the target scope because omission cannot prove absence.

This is deliberately conservative. A deployment can add richer sufficient statistics, such as a signed aggregate summary, if the downstream agent understands that representation. The prototype avoids that extra assumption and keeps the recovered payload as ordinary JSON records.

4.4 Safety property

For a contract K , let A_K be the deterministic answer function associated with the declared task family. The contract is sufficient when equality of its invariant vector implies equality of A_K .

$$\text{If } I(x; K) = I(y; K), \text{ then } A_K(x) = A_K(y).$$

The benchmark contract definitions are constructed to satisfy this implication. The gate therefore gives a direct property for those task families: an accepted or recovered payload cannot change the benchmark answer unless the contract or invariant implementation itself is wrong. This is stronger than textual similarity but narrower than general semantic equivalence.

5. Experimental Design

5.1 Research questions

RQ1. How much structured context can each method remove?

RQ2. How often does each method preserve the exact task answer and declared invariants?

RQ3. Which task families expose silent failures in common structural reduction strategies?

RQ4. What runtime cost does the verification and recovery path add?

RQ5. How sensitive are the reported gate results to candidate-compressor aggressiveness, and how much sampling uncertainty surrounds the observed preservation rates?

5.2 Workload generator

We built a seeded synthetic generator rather than using employer or private operational data. Six domains model recurring shapes in tool-using systems: incident operations, identity events, cloud inventory, issue tracking, commerce orders, and security findings. Each record contains common identifiers, timestamps, states, categories, numeric values, severity, source, ownership fields, and deliberately verbose metadata. Task-specific fields include cost, relation keys, evidence URIs, metric names, region, and detail text.

Payload sizes are 50, 100, 250, and 500 records. Five independent seeds are generated for each domain and size. Each payload is evaluated under all eight task families, producing $6 \times 4 \times 5 \times 8 = 960$ benchmark cases. Rare failures and high numeric values are injected away from array boundaries. Repeated events for the same entity create temporal ambiguity for latest-state tasks.

Table 2. Benchmark task families and what must survive compression.

Task	Question shape	Failure exposed
Lookup	Return exact fields for one record	Target row or non-core field removed
Rare event	Return all failed records	Rare record omitted
Latest state	Return most recent state for an entity	Wrong temporal boundary retained
Aggregation	Return count and sum for a category	Rows or numeric field removed

Task	Question shape	Failure exposed
Join	Return rows sharing a relation key	Join key removed
Negative evidence	Prove no forbidden state exists in scope	Rows dropped, making absence unprovable
Threshold	Return all metric values above threshold	Outlier or metric discriminator removed
Provenance	Return evidence sources for critical records	Evidence URI removed

5.3 Baselines

Raw sends the original payload without reduction.

Fixed projection retains every row but only nine globally chosen core fields. It represents a common engineering optimization in which teams decide that a stable subset of JSON keys is normally enough.

Head-tail sampling retains the first ten and last ten rows. It models boundary-preserving truncation.

Anomaly sampling retains the first and last five rows plus failures, high-severity records, and numeric outliers beyond 2.5 standard deviations. The baseline is inspired by a broad class of boundary-and-anomaly heuristics, including practitioner systems, but it is our own implementation and must not be read as a reproduction of Headroom.

Task-aware unverified reduction uses task parameters to select apparently relevant rows and required fields but performs no post-compression invariant check. It intentionally captures the tempting production design: remove almost everything that does not appear needed, then trust the selection logic.

Pattern-retention guard is a study-owned baseline inspired by protected-pattern safeguards such as Headroom's audit-safe SmartCrusher [16]. It starts from the same generic anomaly candidate, identifies task-derived literal row predicates, splices matching original rows back verbatim, and verifies the protected survivor count. It does not call Headroom and does not reproduce Headroom's statistical selector, compaction, CCR retrieval, or implementation. The baseline isolates a narrower question: how far can protected-row retention go before the task requires a richer executable invariant?

Invariant-preserving gate uses anomaly sampling as its initial candidate. It accepts the candidate when the complete task invariant vector matches and otherwise applies the contract-aware recovery transform described in Section 4. Unlike the pattern-retention baseline, its verifier is not limited to positive row matches or survivor counts.

5.4 Metrics and corruption tests

Serialized-context reduction is computed from canonical compact JSON encoded as UTF-8. Exact task-answer preservation compares the deterministic answer function before and after compression. Invariant preservation compares the complete contract invariant vector. Recovery rate records how often the gate rejects the initial candidate and returns a contract-aware projection instead. Median transformation latency is measured across seven repetitions with Python's high-resolution monotonic clock.

A separate fault-injection set contains 300 generated task instances. Each safe compressed output receives three contract-relevant corruptions: deletion of a participating row, deletion of a required field, and mutation of a required value. This produces 900 corrupted outputs. The test asks only whether the invariant comparison detects the semantic fault.

The experiments ran under Python 3.13.5 on a Linux x86-64 environment exposing five Intel Xeon Platinum 8573C CPUs. No LLM inference occurs in this benchmark. That choice isolates

structural correctness and makes the reported latency a property of the gate rather than a network or model endpoint.

OpenAI ChatGPT assisted with initial benchmark-code drafting. The author reviewed the experiment logic, executed the seeded benchmark, inspected the generated outputs, and retained responsibility for the reported methodology and results.

5.5 Secondary public-data validation

The primary benchmark controls task structure and fault injection but remains synthetic. We therefore added a small secondary dataset built from 24 public issue records visible in the Headroom GitHub issue tracker on September 7, 2026 [15]. Each record contains issue number, title, status, label, actor, event, date, source page, and issue URL. The snapshot contains 19 open and 5 closed records. It contains no private repository content, comments, employer data, or production telemetry.

Eight deterministic tasks exercise lookup, closed-item retrieval, latest state for an actor, status aggregation, actor-based join, negative evidence, an issue-number threshold, and provenance retention. We compare the invariant gate with fixed projection, head-tail sampling, a generic issue-triage sampler, an unverified task-aware reducer, and the study-owned pattern-retention guard. The gate uses the issue-triage sampler as its initial candidate. The pattern guard protects task-derived literal matches. Aggregation and numeric-threshold tasks preserve the full population because the baseline does not encode those obligations as literal predicates. The negative-evidence task instead protects the queried forbidden identifier; because that identifier is absent, the protected set is empty. This setup intentionally exposes the difference between survivor-count protection and a contract that requires complete population evidence. The secondary experiment is deliberately small. Its purpose is to test portability of the failure mode and the distinction between row retention and task equivalence, not to estimate repository-wide performance.

5.6 Sensitivity and uncertainty analysis

The reported invariant gate uses an anomaly candidate that retains the first and last five rows and numeric outliers beyond 2.5 standard deviations. We varied the boundary parameter k over $\{2, 5, 10\}$ and the outlier threshold z over $\{1.5, 2.5, 3.5\}$, then reran all 960 seeded cases for each of the nine configurations. This perturbation changes only the initial candidate. Contracts, recovery transforms, workload seeds, and answer functions remain fixed.

We estimate sampling uncertainty separately from parameter sensitivity. Mean reduction receives a 95% case-level bootstrap interval from 1,000 resamples. Exact-answer preservation receives a Wilson score interval. The same Wilson calculation is reported for the eight-task public snapshot. These intervals describe uncertainty within the evaluated case sets. They do not convert either benchmark into a claim about arbitrary production workloads.

6. Results

Table 3. Overall results across 960 benchmark cases per method.

Method	Mean reduction %	Answer preserved %	Invariants preserved %	Recovery %	Median us
Raw	0.0	100.0	100.0	0.0	0.0
Fixed projection	89.84	37.5	37.5	0.0	167.17
Head-tail sampling	82.13	0.52	0.52	0.0	0.50

Method	Mean reduction %	Answer preserved %	Invariants preserved %	Recovery %	Median us
Anomaly sampling	75.99	40.83	39.79	0.0	164.59
Task-aware unverified	99.76	78.54	75.0	0.0	9.11
Pattern-retention guard	70.51	100.0	100.0	72.71	168.21
Invariant-preserving gate	90.23	100.0	100.0	60.21	163.51

6.1 Reduction versus correctness

The raw payload is the correctness reference. Fixed projection removes 89.84% of serialized context, but it preserves only 37.50% of exact answers because five task families depend on fields outside the globally selected core. Head-tail sampling is smaller by 82.13% yet nearly useless for these tasks, preserving 0.52% of answers. The result is expected once target records move away from array boundaries.

Anomaly sampling performs better on rare-event, threshold, and provenance tasks because those tasks align with its selection heuristic. Across all cases it removes 75.99% and preserves 40.83% of answers. Its weakness appears on aggregations, joins, negative evidence, and most latest-state queries. A row does not need to look anomalous to be essential.

Task-aware unverified reduction is the most aggressive baseline. It removes 99.76% of context on average and preserves 78.54% of exact answers. That number is the dangerous one. Most cases succeed, so sampling a few examples could make the reducer appear safe. It still changes roughly one answer in five.

The pattern-retention guard preserves all 960 seeded answers and invariant vectors, but it reduces serialized context by only 70.51% on average. Its recovery or splice path runs in 72.71% of cases. The cost is structural: it retains complete matching rows, and broad predicates preserve many fields that the task does not consume. For aggregation and threshold cases, the baseline becomes deliberately conservative because row-pattern counts alone do not encode the needed arithmetic or numeric predicate.

The invariant-preserving gate removes 90.23% on average and preserves 100% of benchmark answers and declared invariants. It gives up 9.53 percentage points of reduction relative to the unverified task-aware reducer, but gains 19.72 points over the pattern-retention guard while matching its seeded correctness. The difference comes from verifying task semantics and then projecting only the rows and fields sufficient for the contract.

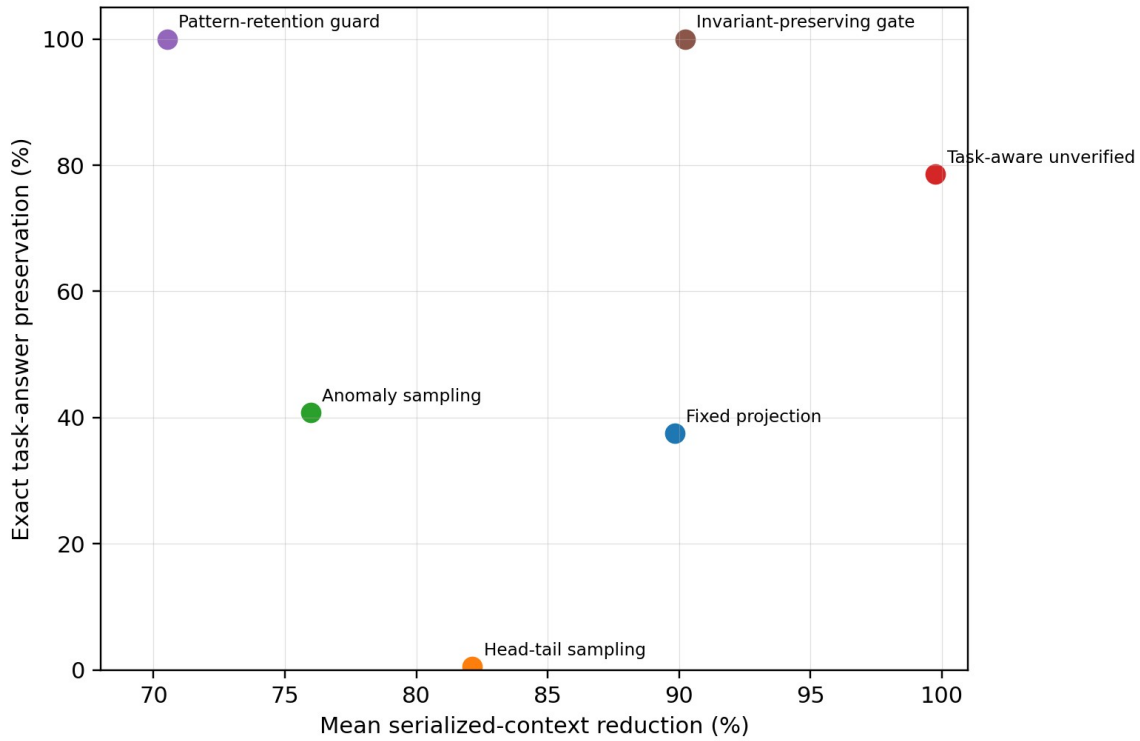


Figure 1. Mean serialized-context reduction versus exact task-answer preservation.

6.2 Failures are task-shaped

Figure 2 separates preservation by task family. The pattern matters more than a single average. Fixed projection succeeds only where the task happens to use its global core fields. Anomaly sampling reaches 100% on rare-event, threshold, and provenance tasks, then collapses on complete-population tasks. The unverified task-aware reducer succeeds on six of eight families but fails on negative evidence and many latest-state cases. The study-owned pattern-retention guard reaches 100% in the seeded workloads by protecting broad task-derived matches, but that safety comes with much lower reduction. The invariant gate reaches the same seeded preservation with a narrower recovered representation.

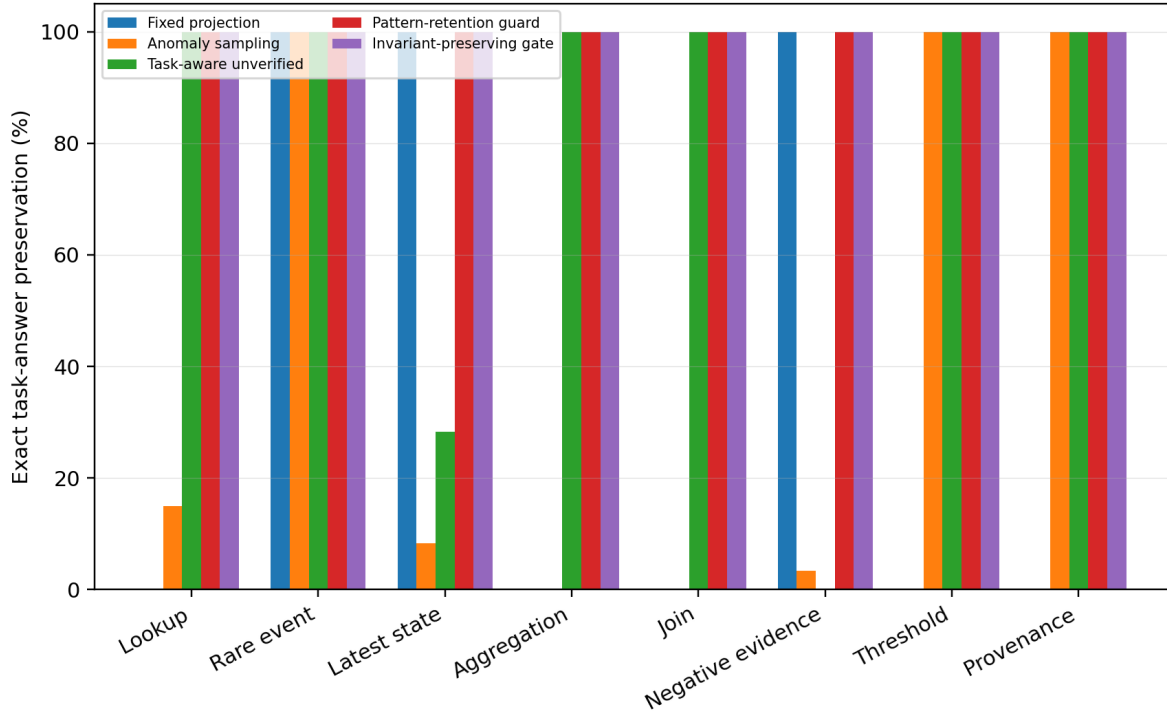


Figure 2. Exact answer preservation by task family. Both guards remain at 100% in the seeded study; their reduction costs differ sharply.

6.3 Recovery is selective

The gate accepts the initial anomaly-sampling candidate without recovery for rare-event, threshold, and provenance tasks in the generated workloads. Those task definitions align with the candidate's anomaly rules. It recovers every aggregation, join, and latest-state case. Lookup requires recovery in 85.00% of cases; negative evidence requires recovery in 96.67%.

Recovery does not imply a return to the full payload. Contract-aware projection still produces strong reductions. Latest-state, aggregation, join, and negative-evidence tasks retain roughly 98.88% to 99.61% reduction in this benchmark after recovery because only a small task-specific row population and field set is required. Rare-event, threshold, and provenance cases remain at the initial candidate's 75.99% mean reduction.

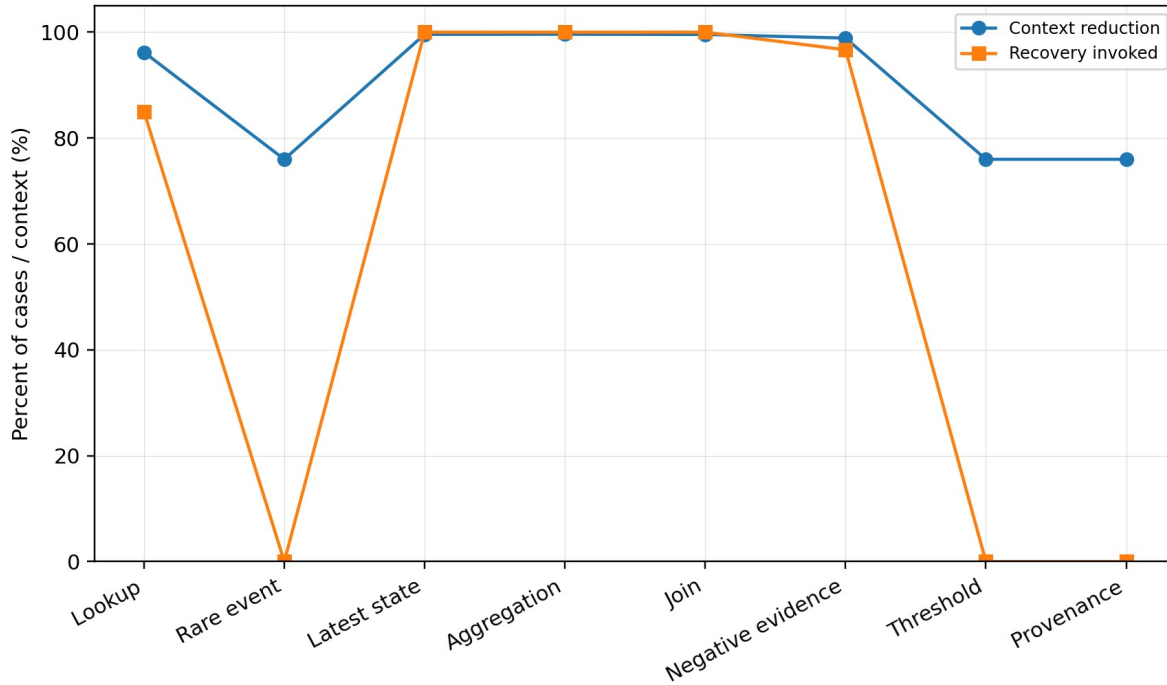


Figure 3. Gate reduction and recovery frequency by task family.

6.4 Fault injection and runtime cost

The invariant checker detected all 900 contract-relevant corruptions. Row deletion, required-field deletion, and required-value mutation were each detected in every injected case. This does not prove that the checker detects arbitrary semantic damage. It confirms that the implemented contracts respond to the failure classes they claim to protect.

Median gate latency was 163.51 microseconds across the 960 cases in the packaged benchmark run. The pattern-retention guard measured 168.21 microseconds, fixed projection 167.17 microseconds, and anomaly sampling 164.59 microseconds in the same Python process. Head-tail sampling was far cheaper because it performs no scan. Timing varied modestly across repeated benchmark executions, as expected at this scale. The measured costs are small relative to remote LLM inference, but they should not be generalized to very large payloads without separate profiling.

6.5 Secondary public-data results

The public issue snapshot sharpens the distinction. The invariant-preserving gate reduced serialized context by 81.47% on average and preserved all eight exact task answers and all declared invariants. The study-owned pattern-retention guard reduced only 26.42% and preserved seven of eight answers. Its failure was the negative-evidence task: the forbidden issue number was absent, so there was no positive matching row to protect, while the generic candidate had already removed population evidence needed to prove absence. The unverified task-aware reducer compressed further, to 88.66%, but preserved only five of eight answers. Fixed projection also preserved five of eight answers. Head-tail sampling preserved none. The issue-triage sampler preserved two of eight answers and only one of eight invariant vectors.

Table 4. Secondary validation on 24 public GitHub issue records across eight tasks.

Method	Mean reduction	Answer preservation	Invariant preservation	Recovery
Raw	0.00%	100.00%	100.00%	0.00%
Fixed projection	39.92%	62.50%	62.50%	0.00%

Method	Mean reduction	Answer preservation	Invariant preservation	Recovery
Head-tail sampling	66.13%	0.00%	0.00%	0.00%
Issue-triage sampler	46.34%	25.00%	12.50%	0.00%
Task-aware unverified	88.66%	62.50%	62.50%	0.00%
Pattern-retention guard	26.42%	87.50%	87.50%	75.00%
Invariant-preserving gate	81.47%	100.00%	100.00%	87.50%

The gate recovered on seven of the eight public-data tasks. That rate is higher than in the seeded study because the generic issue-triage candidate drops records required for exact counts, negative evidence, provenance, and actor-specific queries. The recovery rate is therefore a property of this candidate-data-task combination, not a target operating point.

6.6 Sensitivity and uncertainty

The invariant gate preserved 100% of exact answers and invariant vectors under all nine candidate configurations. Mean reduction ranged from 84.83% at $k=10$ and $z=1.5$ to 92.21% at $k=2$ and $z=3.5$. Recovery frequency stayed within 59.17% to 60.42%. The safety result is therefore insensitive to these candidate settings in the seeded benchmark because failed candidates are rejected by the same external contract. Compression efficiency moves, as expected, because a more aggressive initial candidate changes how often useful rows survive before verification.

Table 5. Sensitivity of invariant gate mean reduction to anomaly-candidate parameters. Every cell preserves 100% of exact answers and declared invariants.

Boundary k	$z=1.5$	$z=2.5$	$z=3.5$	Recovery range
2	89.67%	92.11%	92.21%	59.48-60.42%
5	87.82%	90.23%	90.32%	59.27-60.21%
10	84.83%	87.17%	87.26%	59.17-60.10%

For the reported $k=5$, $z=2.5$ configuration, the 1,000-resample bootstrap interval for mean reduction is 89.42% to 91.00%. The gate's 960-of-960 answer-preservation result has a Wilson 95% interval of 99.60% to 100.00%. The task-aware unverified reducer's 78.54% answer preservation has a Wilson interval of 75.83% to 81.02%. The public snapshot is intentionally weak for estimating a population rate: eight of eight preserved answers correspond to a Wilson interval of 67.56% to 100.00%. That width is a warning against treating the secondary result as a stable deployment percentage.

7. Discussion

7.1 The highest compression ratio is not the right objective

The task-aware baseline demonstrates why a single reduction percentage is a weak success criterion. A 99.76% reduction looks excellent until the 78.54% answer-preservation rate is inspected. The protected-row comparison exposes the opposite failure mode. Conservative row retention can preserve seeded answers while wasting task-irrelevant fields, and it can still fail when correctness depends on absence rather than a positive match. Compression safety therefore needs both a task obligation and a verifier capable of expressing that obligation.

Recent work reaches a similar warning from a different direction. Production-oriented studies report that excessive prompt reduction can increase generated output length or total cost, and TRACE reports execution instability after recurrent context compaction [8,11,12]. Our result adds a structured-data mechanism: a compressor can look efficient while silently changing the set, order, relation, or evidence facts supplied to the model.

7.2 Why a gate should sit outside the compressor

A compressor usually optimizes for one family of signals: redundancy, lexical importance, salience, statistical anomaly, learned summary quality, task attribution, or protected-pattern retention. No one signal covers every structured query. Headroom 0.36.0's audit-safe path is an instructive case [16,17]. It can guarantee survival of rows matching configured patterns and can fail closed if those matches disappear. That is strong protection for known positive evidence. It does not itself state that an exact count must remain unchanged, that a forbidden item must remain absent over a complete scope, that one timestamp is the true maximum, or that a relation spans every required row. A caller could protect the entire population to make some of those tasks safe, but reduction then collapses. By keeping the task contract separate, the same safety boundary can wrap a heuristic library, a learned compressor, a model-generated summary, or a practitioner system.

This separation also makes failures observable. A gate rejection is an event that can be counted by tool, endpoint, task family, or contract. High rejection rates reveal a mismatch between the compressor and the workload. Silent degradation becomes telemetry.

7.3 Contract authoring is the hard part

The verifier is easy to code. Choosing sufficient invariants is harder. Tool owners know which fields form identity, which timestamps define recency, which null values carry meaning, and which collections require completeness. Those rules already exist informally in production services. The proposal is to make them executable at the LLM context boundary.

Automatic contract inference is tempting, but treating a language model's inferred contract as ground truth would reintroduce the uncertainty the gate is intended to remove. A safer path is assisted generation followed by developer review, schema tests, and mutation tests. Contracts should be versioned with the tool schema and exercised in CI.

7.4 Interaction with reversible compression

Reversible systems solve a related but different problem. If an agent can retrieve omitted content later, the cost of an initial mistake may be reduced. Retrieval still depends on the agent realizing that something is missing. An invariant gate acts earlier. It blocks loss of declared facts before inference. The two mechanisms can be combined: reversible compression for broad context reduction, invariant checks for facts that must be present immediately, and retrieval for details whose need cannot be known in advance.

8. Threats to Validity

The controlled benchmark remains synthetic, which limits distributional realism. The secondary experiment reduces that concern by using 24 public GitHub issue records, but it is small, comes from one repository, and exercises only eight deterministic tasks. Its eight-of-eight gate preservation result has a Wilson 95% interval of 67.56% to 100.00%, which is too wide for a general preservation-rate claim. The reported percentages should therefore be read as study-specific measurements, not universal deployment rates.

The study evaluates deterministic task answers, not LLM responses. This choice tests whether the structured evidence required by a declared task survives compression. It does not prove that a specific language model will produce identical natural-language output from two task-equivalent payloads. An external raw-versus-gated model evaluation is supplied as an optional harness in the artifact but is not included in reported results because no external API credential was available in the execution environment.

Serialized UTF-8 bytes are not model tokens. The execution environment did not provide a production tokenizer or an external model API credential. Tokenizers split field names, punctuation, numbers, and repeated text differently. The paper therefore reports exact compact-JSON byte reduction and avoids converting it into a

claimed tokenizer-specific token percentage. The optional model-evaluation harness records API-reported input and output token counts when run externally.

All reported candidate baselines, including the pattern-retention guard, are implemented in this study. The latter intentionally models protected-row retention semantics but is not Headroom. Current Headroom 0.36.0 hard-imports a native Rust-backed SmartCrusher and implements audit-safe row protection in Python around the native selection path [17]. The artifact includes an external comparison harness pinned to that public API. No Headroom runtime result is reported because the prebuilt native wheel could not cross the study environment's package-download boundary. Source inspection is used only to delimit the research claim, not as a substitute for a measured product baseline.

Contracts are manually defined and task-specific. A missing invariant can let a semantically harmful reduction pass. This is not a peripheral issue. Contract sufficiency is the main trust assumption of the method. Mutation testing, versioning, and explicit fail-open behavior reduce that risk but do not erase it.

Latency was measured inside a shared container with five exposed CPUs. Microsecond values will vary by interpreter, hardware, payload schema, and implementation language. The more stable result is the algorithmic observation that the verifier scans structured records and avoids model inference.

9. Practical Deployment Pattern

A production integration can place the gate between tool execution and message construction. The tool returns its native object. The application resolves a contract from the tool name, schema version, and downstream operation. The configured compressor creates a candidate. The gate verifies it. Only the accepted or recovered object is serialized into the message sent to the model.

Three operational metrics should be recorded: compression reduction, gate rejection rate, and fail-open rate. Reduction alone rewards unsafe behavior. Rejection rate shows where a compressor disagrees with application semantics. Fail-open rate exposes contract or recovery bugs that require engineering attention.

Candidate aggressiveness can be tuned independently of the safety boundary. In the seeded sensitivity sweep, changing k and z moved mean reduction by more than seven percentage points while the external invariant check held answer preservation constant. This separation is operationally useful: teams can search for a cheaper or faster compressor without silently changing the declared correctness condition.

Cost accounting should include output tokens and retries, not just the first compressed input. Recent preprints show that aggressive compression can trigger longer outputs in some settings [11,12]. The gate does not solve that economic problem by itself. It removes one cause of uncertainty: known structured facts do not disappear silently when the contract is sufficient.

10. Conclusion

Structured tool outputs permit a stronger compression safety rule than semantic similarity alone. The application can state executable invariants, test a compressed candidate against the original, recover task-required rows and fields when the candidate fails, and return the original payload when recovery cannot prove equivalence. Across 960 seeded benchmark cases, this gate reduced serialized context by 90.23% while preserving every declared invariant and exact benchmark answer. The result held at 100% preservation across nine candidate-aggressiveness settings, with mean reduction ranging from 84.83% to 92.21%. A study-owned protected-row guard also preserved every seeded answer but reduced context by only 70.51%, while an unverified task-aware

reducer reached 99.76% reduction and changed 21.46% of answers. On a separate 24-record public issue snapshot, the invariant gate reduced serialized context by 81.47% while preserving all eight task answers and invariants. The small public sample has wide statistical uncertainty and is not offered as a universal rate. The result is narrow by design. It does not claim universal semantic equivalence, tokenizer-specific savings, model-independent output identity, or a reproduced Headroom runtime result. It shows that protected-row retention and task-equivalence verification operate at different levels of the compression-safety problem.

Data and Code Availability

The reproducibility package accompanying this manuscript contains the seeded benchmark generator, 6,720 method-case observations, task-level aggregates, 900 corruption tests, figure-generation code, the study-owned pattern-retention baseline, the 24-record public GitHub issue snapshot, the eight-task secondary-validation script, and its raw results. It also contains the nine-setting sensitivity sweep, bootstrap and Wilson interval outputs, and an external Headroom 0.36.0 harness aligned to the public Rust-backed SmartCrusher API. The package includes an optional external-LLM evaluation harness that is not part of the reported results. The primary benchmark is synthetic; the secondary snapshot contains only public issue-list metadata. No employer data, private logs, proprietary implementation details, or production records are included. A permanent archival URL or DOI should replace this statement's package reference before final publication.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used OpenAI ChatGPT to assist with literature organization, experimental code drafting, manuscript drafting, and language editing. The author reviewed and edited the generated material, reran the reported experiments, inspected the resulting data, verified cited sources, and takes full responsibility for the content of the publication.

References

1. Li Z, Liu Y, Su Y, Collier N. Prompt Compression for Large Language Models: A Survey. *Proceedings of NAACL-HLT 2025*. 2025:7182-7195. doi:10.18653/v1/2025.naacl-long.368.
2. Jiang H, Wu Q, Lin CY, Yang Y, Qiu L. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. *Proceedings of EMNLP 2023*. 2023:13358-13376. doi:10.18653/v1/2023.emnlp-main.825.
3. Jiang H, Wu Q, Luo X, Li D, Lin CY, Yang Y, Qiu L. LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *Proceedings of ACL 2024*. 2024:1658-1677. doi:10.18653/v1/2024.acl-long.91.
4. Li Z, Su Y, Collier N. 500xCompressor: Generalized Prompt Compression for Large Language Models. *Proceedings of ACL 2025*. 2025:25081-25091. doi:10.18653/v1/2025.acl-long.1219.
5. Xu Z, Li F, Chen AC, Wang X. ProCut: LLM Prompt Compression via Attribution Estimation. *Proceedings of EMNLP 2025 Industry Track*. 2025:285-309. doi:10.18653/v1/2025.emnlp-industry.20.

6. Łajewska W, Hardalov M, Aina L, John NA, Su H, Márquez L. Understanding and Improving Information Preservation in Prompt Compression for LLMs. Findings of EMNLP 2025. 2025:17520-17541. doi:10.18653/v1/2025.findings-emnlp.949.
7. Kang M, Chen WN, Han D, Inan H, Wutschitz L, Chen Y, Sim R, Rajmohan S. ACON: Optimizing Context Compression for Long-horizon LLM Agents. International Conference on Machine Learning. 2026.
8. Min G, Wu L, Darbari M, Chen C, Hong L. Toward Reliable Context Compression for Long-Horizon Agents: An Empirical Study of Execution Instability. arXiv:2608.06503. 2026. doi:10.48550/arXiv.2608.06503.
9. Qin Y, Liang S, Ye Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. International Conference on Learning Representations. 2024. arXiv:2307.16789.
10. Meyer B. Applying Design by Contract. Computer. 1992;25(10):40-51. doi:10.1109/2.161279.
11. Johnson W, Lee C. Prompt Compression in Production Task Orchestration: A Pre-Registered Randomized Trial. arXiv:2603.23525. 2026. doi:10.48550/arXiv.2603.23525.
12. Johnson W. Compression Method Matters: Benchmark-Dependent Output Dynamics in LLM Prompt Compression. arXiv:2603.23527. 2026. doi:10.48550/arXiv.2603.23527.
13. Headroom Labs AI. Headroom: The Context Optimization Layer for LLM Applications. GitHub repository. <https://github.com/headroomlabs-ai/headroom>. Accessed September 7, 2026.
14. Headroom Labs AI. Headroom Benchmarks: Compression Performance, Accuracy Preservation, and Latency Overhead. <https://github.com/headroomlabs-ai/headroom/blob/main/docs/content/docs/benchmarks.mdx>. Accessed September 7, 2026.
15. Headroom Labs AI. Issues. GitHub repository issue tracker. <https://github.com/headroomlabs-ai/headroom/issues>. Accessed September 7, 2026.
16. Headroom Labs AI. feat(compression): add audit-safe mode with protected pattern matching. GitHub Pull Request #1899. Merged July 9, 2026. <https://github.com/headroomlabs-ai/headroom/pull/1899>. Accessed September 7, 2026.
17. Headroom Labs AI. SmartCrusher source, headroom v0.36.0. GitHub repository. https://github.com/headroomlabs-ai/headroom/blob/v0.36.0/headroom/transforms/smart_crusher.py. Accessed September 7, 2026.